جامعة الحسن الثاني بالدار البيضاء
كلية العلوم بن مسيك
FSBM
FACULTE DES SCIENCES BEN M'SICK
UNIVERSITÉ HASSAN II DE CASABLANCA

TD/TP : Entraînement Distribué avec PyTorch

Objectif du TP

Analyser l'influence de la distribution du calcul sur l'efficacité de l'entraînement d'un modèle de Deep Learning. À travers l'entraînement de ResNet-50 sur CIFAR-10 selon trois configurations matérielles progressives (CPU → 1 GPU → 2 GPU DataParallel), cette étude vise à quantifier les gains de performance obtenus à chaque niveau de parallélisation.

Pré-requis techniques

Plateforme : Kaggle (GPU T4 recommandé — sélectionner 2× T4 si disponible).

Librairies Python requises :

- Python
- PyTorch 2.0
- matplotlib / numpy

Avant de lancer : Exécution → Modifier le type d'exécution GPU T4 (Kaggle : Settings → Accelerator → GPU T4 × 2)

Vérification du numéro de téléphone requise : Pour activer l'accélérateur GPU sur Kaggle, vous devez au préalable vérifier votre numéro de téléphone. Rendez-vous dans Settings → Phone Verification et suivez les étapes de validation. Sans cette vérification, l'option GPU restera désactivée.

Plan en 7 étapes

Étape	Description	Config
1	Préparation de l'environnement	
2	Exploration et visualisation du dataset CIFAR-10	
3	Définition du modèle ResNet-50	
4	Entraînement sur CPU	Config A
5	Entraînement sur 1 GPU	Config B
6	Entraînement DataParallel (2 GPU, distribué)	Config C
7	Comparaison et Visualisation des résultats	A vs B vs C

Contexte

Dans ce projet, nous utilisons ResNet pré-entraîné sur ImageNet afin de réaliser une classification sur CIFAR-10.

ImageNet est une grande base de données contenant des millions d'images utilisées pour entraîner des modèles de Deep Learning. Les modèles pré-entraînés sur ImageNet apprennent déjà des caractéristiques importantes des images comme les formes, les contours et les textures, ce qui permet d'accélérer l'apprentissage et d'améliorer les performances.

- Les images CIFAR-10 de taille : 32×32
- sont redimensionnées en : 224×224
- pour être compatibles avec ResNet-50

```
# — Transforms ————————————————————————————
transform_train = transforms.Compose([
    transforms.Resize(224),
    transforms.RandomHorizontalFlip(),
    transforms.RandomCrop(224, padding=8),
    transforms.ColorJitter(brightness=0.2, contrast=0.2,
                           saturation=0.2, hue=0.1),
    transforms.ToTensor(),
    transforms.Normalize(mean=(0.4914, 0.4822, 0.4465),
                         std =(0.2023, 0.1994, 0.2010)),
])
transform_test = transforms.Compose([
    transforms.Resize(224),
    transforms.ToTensor(),
    transforms.Normalize(mean=(0.4914, 0.4822, 0.4465),
                         std =(0.2023, 0.1994, 0.2010)),
])

# — Téléchargement CIFAR-10 ———————————————————————
print('Téléchargement CIFAR-10...')
train_dataset_full = torchvision.datasets.CIFAR10(
    root=DATA_DIR, train=True,  download=True, transform=transform_train
)
test_dataset = torchvision.datasets.CIFAR10(
    root=DATA_DIR, train=False, download=True, transform=transform_test
)
```

Partie 1 : Préparation et Exploration

Étape 1 — Préparation de l'environnement

Vérifiez les ressources GPU disponibles et configurez le device principal.

À implémenter :

- Importer les librairies (torch, torchvision, matplotlib, numpy…)
- Afficher torch.__version__, torch.cuda.is_available(), torch.cuda.device_count()
- Lister les GPU avec torch.cuda.get_device_properties(i)
- Limiter à 2 GPU maximum (variable NUM_GPU)

```
================================================================
   ÉTAPE 1 — Préparation de l'environnement
================================================================
PyTorch version  : 2.10.0+cu128
CUDA disponible  : True
Nombre de GPU    : 2
  GPU 0 : Tesla T4 — 15.6 GB VRAM
  GPU 1 : Tesla T4 — 15.6 GB VRAM

GPU utilisés pour le distribué : 2
================================================================
```

Étape 2 — Exploration des données

Chargez le dataset CIFAR-10 **sans transformation** pour en explorer la structure avant tout preprocessing.

À implémenter :

- Charger torchvision.datasets.CIFAR10 (train + test, download=True)
- Récupérer les noms des classes (dataset.classes)
- Inspecter la première image : taille , label, type
- Afficher 10 images en grille avec matplotlib et leurs étiquettes

Étape 3 — Définition du modèle ResNet-50

Adaptez ResNet-50 pré-entraîné à la classification CIFAR-10 (10 classes) en ne remplaçant que la couche fully-connected finale.

À implémenter :

- Charger models.resnet50(pretrained=True) ou weights=ResNet50_Weights.DEFAULT
- Remplacer model.fc par un nn.Linear(2048, 10)
- Afficher le nombre de paramètres totaux et entraînables
- Définir **un seul paramètre** SUBSET = 10000 qui contrôle la taille du sous-ensemble

Hyper-paramètres communs aux 3 configs : BATCH_SIZE=256, NUM_EPOCHS=4, NUM_WORKERS=2, SUBSET=10000

Partie 2 : Les trois configurations d'entraînement

Config A — Entraînement sur CPU

Entraînez ResNet-50 sur CPU **sans aucune accélération**. Cette configuration sert de baseline absolue. L'entraînement sera lent (2 h 25 min 48 s sur 4 epochs).

À implémenter :

- Définir device = torch.device('cpu')
- Créer le DataLoader
- Boucle d'entraînement standard (forward □ loss □ backward □ optimizer.step)
- Mesurer le temps par epoch avec time.time()
- Stocker : train_loss, val_loss, val_acc, ep_times

```
================================================================
   ÉTAPE 4 — CONFIG A : Entraînement sur CPU (benchmark)
================================================================
  Sous-ensemble : 10000 images | batch=32 | 4 epochs

----------------------------------------------------------------
  Configuration : CPU
  Device        : cpu
  Mixed Prec.   : Non (float32)
----------------------------------------------------------------

  — Epoch 1/4
    [CPU] Batch  100/313 | Loss 1.6279 | Acc 54.3%
    [CPU] Batch  200/313 | Loss 1.2785 | Acc 68.5%
    [CPU] Batch  300/313 | Loss 1.1384 | Acc 74.3%
  → Loss:1.1257 | Train:74.9% | Val:0.0% | Temps:2238.5s

  — Epoch 2/4
    [CPU] Batch  100/313 | Loss 0.7666 | Acc 90.1%
    [CPU] Batch  200/313 | Loss 0.7554 | Acc 90.3%
    [CPU] Batch  300/313 | Loss 0.7494 | Acc 90.7%
  → Loss:0.7472 | Train:90.8% | Val:0.0% | Temps:2198.9s

  — Epoch 3/4
    [CPU] Batch  100/313 | Loss 0.6697 | Acc 94.5%
    [CPU] Batch  200/313 | Loss 0.6595 | Acc 94.8%
    [CPU] Batch  300/313 | Loss 0.6555 | Acc 95.0%
  → Loss:0.6558 | Train:95.0% | Val:0.0% | Temps:2162.4s

  — Epoch 4/4
    [CPU] Batch  100/313 | Loss 0.6183 | Acc 96.5%
    [CPU] Batch  200/313 | Loss 0.6147 | Acc 96.7%
    [CPU] Batch  300/313 | Loss 0.6118 | Acc 96.9%
  → Loss:0.6113 | Train:96.9% | Val:0.0% | Temps:2145.3s

  Terminé en 145.8 min (moy. 2186.3s/epoch)

  Config A (CPU) terminée
```

Config B — Entraînement sur 1 GPU avec AMP

Ajoutez **l'accélération GPU** et **Automatic Mixed Precision (AMP)** pour réduire la consommation mémoire et accélérer les calculs en précision float16.

À implémenter :

- Définir device = torch.device('cuda:0')
- Transférer le modèle sur GPU : model.to(device)
- Activer AMP : scaler = torch.cuda.amp.GradScaler()
- Utiliser torch.autocast('cuda') dans la boucle forward
- Remplacer loss.backward() par scaler.scale(loss).backward()

(autocast accélère les calculs en float16, et scaler protège les gradients pour qu'ils ne disparaissent pas.)

```
================================================================
   ÉTAPE 5 — CONFIG B : Entraînement sur 1 GPU
================================================================

----------------------------------------------------------------
  Configuration : 1 GPU
  Device        : cuda:0
  Mixed Prec.   : Oui (float16)
----------------------------------------------------------------

  ── Epoch 1/4
  → Loss:2.0422 | Train:38.4% | Val:71.0% | Temps:67.6s

  ── Epoch 2/4
  → Loss:1.2955 | Train:72.3% | Val:83.9% | Temps:66.6s

  ── Epoch 3/4
  → Loss:0.9980 | Train:81.8% | Val:87.1% | Temps:65.1s

  ── Epoch 4/4
  → Loss:0.9113 | Train:85.3% | Val:87.9% | Temps:64.9s

  Terminé en 4.4 min (moy. 66.0s/epoch)
  Meilleure val acc : 87.89%

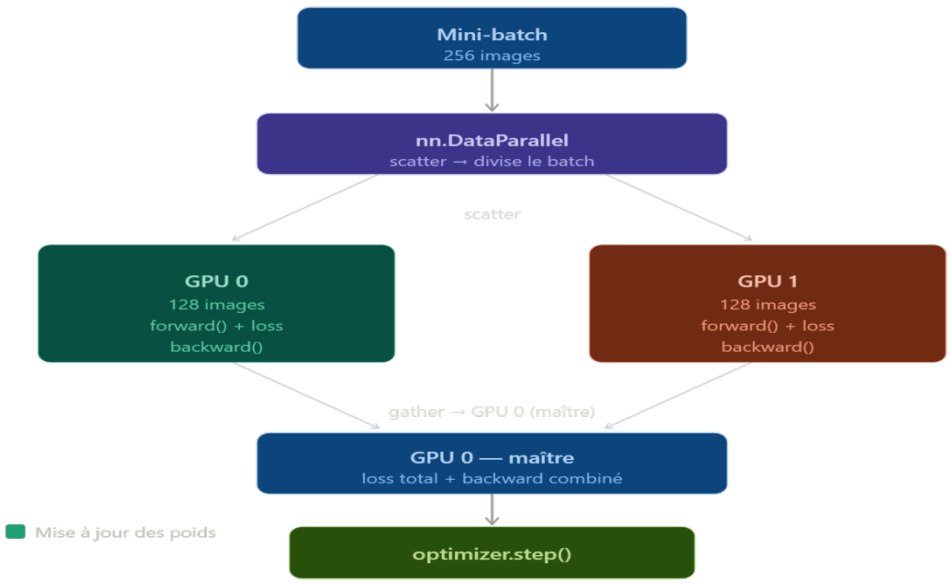
  Config B (1 GPU) terminée
```

Config C — DataParallel (2 GPU, entraînement distribué)

« **DataParallel** est une technique qui permet de diviser le travail d'entraînement entre **plusieurs GPU** en même temps pour aller plus vite.»

Exploitez les **2 GPU disponibles** via nn.DataParallel, la méthode la plus simple de parallélisme de données en PyTorch.

Concept — fonctionnement de DataParallel :



À implémenter :

- Envelopper le modèle : model = nn.DataParallel(model, device_ids=[0, 1])
- Transférer sur GPU master : model.to('cuda:0')

```
======================================================================
  ÉTAPE 6 — CONFIG C : Entraînement Distribué (DataParallel)
======================================================================
  DataParallel activé : 2 GPU(s) → 128 images/GPU

----------------------------------------------------------------------
  Configuration : DataParallel (2 GPU)
  Device        : cuda
  Mixed Prec.   : Oui (float16)
  DataParallel  : 2 GPU(s)
----------------------------------------------------------------------

  ── Epoch 1/4
  → Loss:2.0626 | Train:38.2% | Val:67.5% | Temps:54.9s

  ── Epoch 2/4
  → Loss:1.3061 | Train:72.0% | Val:84.6% | Temps:54.3s

  ── Epoch 3/4
  → Loss:0.9847 | Train:82.9% | Val:87.7% | Temps:54.0s

  ── Epoch 4/4
  → Loss:0.9179 | Train:85.2% | Val:88.5% | Temps:54.3s

  Terminé en 3.6 min (moy. 54.4s/epoch)
  Meilleure val acc : 88.45%

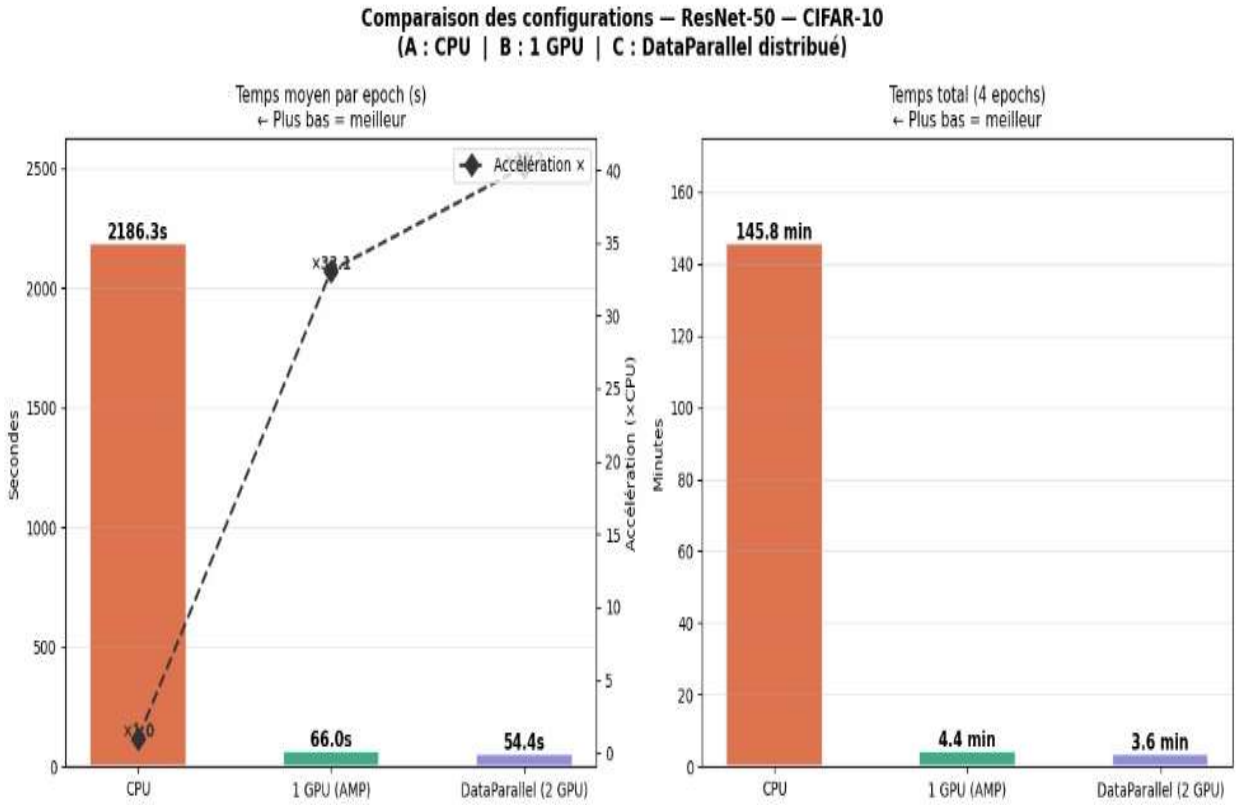
  Config C (DataParallel) terminée
```

Partie 3 : Comparaison et Visualisation

Après les 3 entraînements, produisez des **graphiques** :

Graphique 1 — Comparaison des temps d'entraînement :

Q1. En utilisant les résultats des 3 configurations (CPU, 1 GPU, 2 GPU), produisez un **bar chart** comparant le **temps moyen par epoch** et le **temps total** pour chaque configuration.

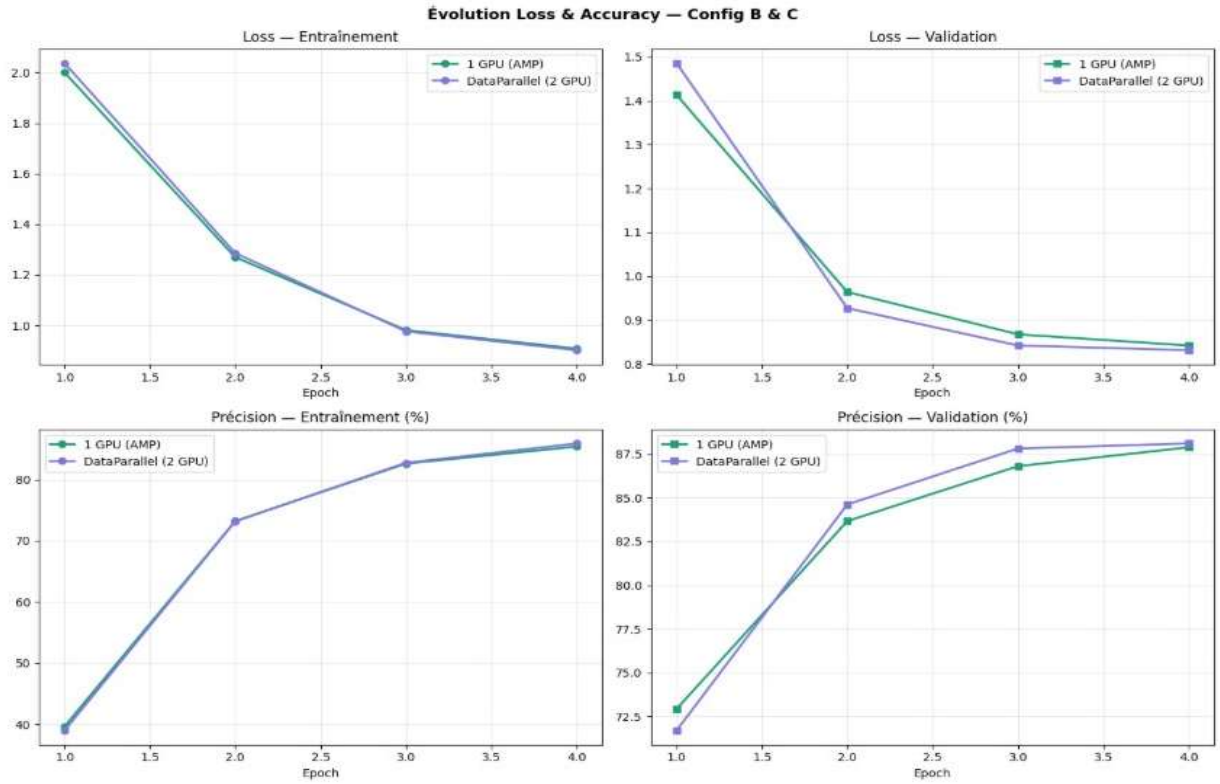


Graphique 2 — Courbes Loss et Accuracy :

Q1. Tracez sur un même graphique (4 sous-figures) les courbes de **train loss**, **val loss**, **train accuracy** et **val accuracy** pour les configurations **1 GPU** et **2 GPU** sur les 4 epochs.

Q2. Les deux courbes (1 GPU et 2 GPU) sont quasi identiques. Que cela démontre-t-il sur l'effet de DataParallel sur la **qualité d'apprentissage** ? Justifiez en expliquant comment DataParallel **synchronise les gradients** entre les deux GPU.

Q3. La précision de validation est légèrement supérieure avec 2 GPU (88.45%) par rapport à 1 GPU (87.89%). Quelle en est la raison liée au **batch effectif** ?



Q4. Complétez le tableau récapitulatif suivant avec vos résultats mesurés :

Configuration	Moy. epoch	Total	Val Acc
CPU			—
1 GPU (AMP)			
DataParallel (2 GPU)			

Bon Courage 😊